

사이버보안개론

Introduction to Cybersecurity

유준수 (Joon Soo Yoo, Ph.D.)

Fall 2026

Defense & Advanced Security Lab (DAS Lab)

Department of Advanced Defense Technology and Industry

Jeonbuk National University

Chapter 2. Cryptographic Tools

2.2 Message Authentication and Hash Functions

Integrity and Authentication

Learning Objectives

이번 Section에서는 Message Authentication과 Cryptographic Hash Function의 기본 개념을 학습한다.

Learning Goals

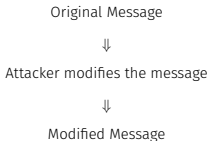
- ▶ Confidentiality와 Message Authentication의 차이를 이해한다.
- ▶ Message Authentication Code (MAC)의 역할을 이해한다.
- ▶ Hash Function과 MAC의 차이를 이해한다.
- ▶ Secure Hash Function의 주요 Security Property를 이해한다.
- ▶ Preimage, Second Preimage, Collision Resistance를 구분할 수 있다.
- ▶ Birthday Paradox와 Collision Security의 관계를 이해한다.
- ▶ SHA 계열의 기본적인 발전 흐름을 이해한다.

1. Why Message Authentication?

앞에서는 Encryption을 이용하여 Data의 Confidentiality를 보호하였다.

Attacker reads the message → Encryption

하지만 공격자가 Message를 읽는 것뿐 아니라 변경할 수도 있다.



따라서 다음과 같은 질문도 필요하다.

- ▶ Message가 중간에 변경되지 않았는가?
- ▶ Message가 실제 Sender로부터 왔는가?

핵심

Encryption은 주로 Confidentiality를 제공하고, Message Authentication은 Integrity와 Source Authentication을 확인한다.

1-1. What Should Be Verified?

Message Authentication에서는 Received Message가 Authentic한지 확인한다.

주요 확인 대상은 다음과 같다.

- ▶ Integrity

Message의 내용이 변경되지 않았는가?

- ▶ Source Authentication

Message가 주장된 Sender로부터 왔는가?

- ▶ Timeliness

오래된 Message가 지연되거나 Replay된 것은 아닌가?

- ▶ Sequence

Message의 순서가 변경되지 않았는가?

핵심

Message Authentication은 Message의 내용뿐 아니라 Source, Time, Sequence와 관련된 문제도 다룰 수 있다.

1-2. Is Encryption Alone Enough?

Encryption된 Message가 정상적으로 Decryption된다고 해서 Message가 변조되지 않았다고 보장할 수는 없다.

ECB Mode를 생각해보자.

$$C_1 = E_K(P_1), \quad C_2 = E_K(P_2), \quad C_3 = E_K(P_3)$$

공격자가 Ciphertext Block의 순서를 바꾸면

$$C_1, C_2, C_3 \longrightarrow C_3, C_1, C_2$$

Receiver는 여전히 각 Block을 정상적으로 Decryption할 수 있다.

$$P_3, P_1, P_2$$

하지만 전체 Message의 의미는 변경될 수 있다.

중요

Successful Decryption이 Message Integrity를 보장하는 것은 아니다.

2. Message Authentication Code (MAC)

Message Authentication을 위한 대표적인 방법이 Message Authentication Code (MAC)이다.

Sender와 Receiver가 Secret Key K 를 공유한다고 하자.

Sender는 Message M 에 대해

$$T = MAC_K(M)$$

을 계산한다.

그리고 다음을 전송한다.

$$M \parallel T$$

여기서 T 는 Message에 대한 Authentication Tag이다.

핵심

MAC은 Message와 Secret Key를 이용하여 Message를 검증하기 위한 Authentication Tag를 생성한다.

2-1. Verifying a MAC

Receiver도 동일한 Secret Key K 를 가지고 있다.

Received Message M 에 대해

$$T' = \text{MAC}_K(M)$$

을 다시 계산한다.

그리고 Received Tag T 와 비교한다.

$$T' \stackrel{?}{=} T$$

Match $\rightarrow$ Accept

Not Match $\rightarrow$ Reject

공격자가 Message를 M' 으로 변경하면 올바른 Secret Key 없이

$$\text{MAC}_K(M')$$

을 생성할 수 없어야 한다.

핵심

MAC Verification을 통해 Message Integrity와 Source Authentication을 확인할 수 있다.

2-2. Authentication vs. Confidentiality

MAC 자체는 Message를 암호화하지 않는다.

$$M \parallel MAC_k(M)$$

을 전송하면 공격자는 M 을 읽을 수 있다.

Different Security Goals

	Confidentiality	Integrity / Authentication
Encryption	Yes	Not necessarily
MAC	No	Yes

Confidentiality와 Authentication이 모두 필요하다면 Encryption과 Authentication을 함께 사용할 수 있다.

핵심

Encryption과 Message Authentication은 서로 다른 Security Goal을 제공한다.

3. What is a Hash Function?

Hash Function은 Variable-Length Message를 입력받아 Fixed-Length Hash Value를 생성한다.

$$h = H(M)$$

Message M



Hash Function H



Hash Value h

Hash Value는 다음과 같이 부르기도 한다.

- ▶ Hash
- ▶ Message Digest
- ▶ Fingerprint

핵심

Hash Function은 크기가 다양한 Data를 고정된 크기의 Hash Value로 변환한다.

3-1. Hash Function vs. MAC

Hash Function과 MAC의 중요한 차이는 Secret Key의 사용 여부이다.

Hash:

$$h = H(M)$$

MAC:

$$T = MAC_K(M)$$

Hash vs. MAC

	Hash	MAC
Input	Message	Message + Secret Key
Secret Key	No	Yes
Fixed-Size Output	Yes	Yes
Authentication	By itself, No	Yes

핵심

Hash Function 자체에는 Secret Key가 없기 때문에 Hash만으로 Sender를 Authentication할 수는 없다.

3-2. Why Is Hash Alone Not Authentication?

Sender가 다음을 전송한다고 생각해보자.

$$M \parallel H(M)$$

공격자가 Message를

$$M \rightarrow M'$$

으로 변경할 수 있다.

Hash Function은 공개되어 있으므로 공격자도 새로운 Hash를 계산할 수 있다.

$$H(M')$$

그리고

$$M' \parallel H(M')$$

을 Receiver에게 전송할 수 있다.

Receiver는 Hash만으로 이러한 공격을 구분할 수 없다.

중요

Hash Function은 Message의 Fingerprint를 만들지만, Hash만으로는 Message Authentication을 제공하지 않는다.

3-3. Authentication Using a Hash Function

Hash Function에 Secret Key를 결합하면 Message Authentication에 사용할 수 있다.

교재의 기본적인 예를 생각하면,

$$T = H(K \parallel M \parallel K)$$

와 같은 형태를 사용할 수 있다.

Sender와 Receiver만 Secret Key K 를 알고 있으므로 공격자는 Message를 변경한 후 올바른 Tag를 생성하기 어려워야 한다.

실제 표준적인 방법으로는 HMAC이 널리 사용된다.

핵심

Secret Key와 Hash Function을 결합하여 Message Authentication Code를 구성할 수 있다.

3-4. From Hash to HMAC

Hash Function 자체는 Secret Key를 사용하지 않는다.

$$h = H(M)$$

따라서 Hash만으로는 Message Authentication을 제공할 수 없다.

Secret Key를 Hash Function과 결합하면 MAC을 구성할 수 있다.

$$T = H(K \parallel M \parallel K)$$

이러한 방식을 Keyed Hash MAC이라고 할 수 있다.

현대적으로 널리 사용되는 표준적인 방법은 HMAC (Hash-based Message Authentication Code)이다.

Hash $\xrightarrow{+ \text{ Secret Key }}$ Keyed Hash MAC $\xrightarrow{\text{Standard Construction}}$ HMAC

HMAC은 단순히 Key와 Message를 연결하는 것이 아니라, 안전성을 고려하여 정의된 별도의 구조를 사용한다.

핵심

Hash 자체는 Authentication을 제공하지 않는다. Secret Key와 Hash를 안전하게 결합한 표준적인 Message Authentication 방식이 HMAC이다.

3-5. (참고) HMAC Structure

HMAC은 Secret Key와 Hash Function을 단순히 Concatenation하는 것보다 구조화된 방식을 사용한다.

먼저 Inner Hash를 계산한다.

$$\text{Inner} = H((K' \oplus \text{ipad}) \parallel M)$$

그 결과를 다시 Outer Hash의 Input으로 사용한다.

$$\text{HMAC}_K(M) = H((K' \oplus \text{opad}) \parallel \text{Inner})$$

여기서,

- ▶ K' : Hash Block Size에 맞게 처리된 Secret Key
- ▶ ipad : Inner Padding Constant
- ▶ opad : Outer Padding Constant
- ▶ M : Message

즉 HMAC은

Inner Hash $\longrightarrow$ Outer Hash

의 두 단계 구조를 사용한다.

4. Secure Hash Functions

Cryptographic Hash Function은 단순히 Fixed-Length Output을 만드는 것만으로 충분하지 않다.

대표적인 Security Property는 다음과 같다.

1. Preimage Resistance

Hash Value로부터 원래 Input을 찾기 어려워야 한다.

2. Second Preimage Resistance

주어진 Message와 같은 Hash를 가지는 다른 Message를 찾기 어려워야 한다.

3. Collision Resistance

같은 Hash를 가지는 어떤 두 Message를 찾는 것도 어려워야 한다.

핵심

Secure Hash Function은 One-Way Property와 Collision에 대한 저항성을 가져야 한다.

4-1. Preimage Resistance

Hash Value h 가 주어졌다고 하자.

공격자의 목표는

$$H(x) = h$$

를 만족하는 Input x 를 찾는 것이다.

$$\begin{array}{c} x \\ \Downarrow \\ H(x) \end{array}$$

Forward Direction은 쉽게 계산할 수 있지만,

$$h \rightarrow x$$

와 같은 Reverse Direction은 현실적으로 어려워야 한다.

이를 **Preimage Resistance** 또는 **One-Way Property**라고 한다.

핵심

특정 Hash Value가 주어졌을 때 그 Hash를 만드는 Input을 찾기 어려워야 한다.

4-2. Second Preimage Resistance

이번에는 특정 Message x 가 주어진다.

공격자의 목표는

$$y \neq x$$

이면서

$$H(y) = H(x)$$

를 만족하는 다른 Message y 를 찾는 것이다.

즉,

$$x \longrightarrow H(x)$$

와 동일한 Hash를 가지는 다른 Message를 찾아서는 안 된다.

핵심

주어진 Message와 동일한 Hash를 가지는 다른 Message를 찾기 어려워야 한다.

4-3. Collision Resistance

Collision Attack에서는 특정 Message나 Hash Value가 미리 주어지지 않는다.

공격자는 아무 두 Message x, y 를 찾아서

$$x \neq y$$

이면서

$$H(x) = H(y)$$

가 되도록 하면 된다.

즉 공격자는

$$(x, y)$$

두 Input을 모두 자유롭게 선택할 수 있다.

이 점이 Preimage Attack과 Collision Attack의 중요한 차이이다.

핵심

Collision Resistance는 같은 Hash를 가지는 어떤 두 Message라도 찾기 어려워야 한다는 Property이다.

5. Security Strength of Hash Functions

n -bit Hash Function은 최대

$$2^n$$

개의 서로 다른 Hash Value를 가질 수 있다.

하지만 공격의 종류에 따라 필요한 계산량은 서로 다르다.

Ideal n -bit Hash Function

Attack	Approximate Work
Preimage	2^n
Second Preimage	2^n
Collision	$2^{n/2}$

Collision Attack은 특정 Hash Value를 찾는 것이 아니라, 같은 Hash를 가지는 아무 두 Input을 찾으면 된다.

핵심

Hash Output Length가 n bit라고 해서 모든 공격에 대해 n -bit Security를 제공하는 것은 아니다.

5-1. Why Does Collision Require $2^{n/2}$?

Collision에서는 특정 Hash Value가 목표가 아니다.

공격자가 q 개의 서로 다른 Message를 만들었다고 하자.

$$M_1, M_2, \dots, M_q$$

각 Message의 Hash를 계산하면

$$H(M_1), H(M_2), \dots, H(M_q)$$

를 얻는다.

이 Hash들 사이에서 비교할 수 있는 Pair의 수는

$$\binom{q}{2} = \frac{q(q-1)}{2} \approx \frac{q^2}{2}$$

이다.

즉 Message의 수가 증가하면 비교 가능한 Pair의 수는 약 q^2 으로 증가한다.

핵심

Collision은 하나의 정해진 Hash를 맞추는 문제가 아니라, 많은 Hash 중 같은 값을 가지는 Pair를 찾는 문제이다.

6. Secure Hash Algorithms

대표적인 Cryptographic Hash Function으로 Secure Hash Algorithm (SHA) 계열이 있다.

SHA Family

Algorithm	Hash Output Length
SHA-1	160 bits
SHA-256	256 bits
SHA-384	384 bits
SHA-512	512 bits

SHA-256, SHA-384, SHA-512 등은 SHA-2 Family에 속한다.

SHA-3는 SHA-1 및 SHA-2와 다른 구조를 사용하는 Hash Function Family이다.

핵심

SHA Algorithm 이름의 숫자는 일반적으로 생성되는 Hash Value의 길이를 나타낸다.

6-1. SHA-1, SHA-2, and SHA-3

SHA 계열은 시간이 지나면서 새로운 Algorithm으로 발전해왔다.

SHA-1



SHA-2



SHA-3

- ▶ **SHA-1**
160-bit Hash Value를 생성한다.
- ▶ **SHA-2**
SHA-256, SHA-384, SHA-512 등의 Algorithm을 포함한다.
- ▶ **SHA-3**
SHA-1과 SHA-2와는 다른 내부 구조를 사용하는 별도의 Hash Function Family이다.

핵심

Secure Hash Algorithm도 Security 요구와 Cryptanalysis의 발전에 따라 새로운 표준이 개발되어 왔다.

7. Other Applications of Hash Functions

Cryptographic Hash Function은 Message Authentication 이외에도 다양한 Security Mechanism에서 사용된다.

대표적인 예는 다음과 같다.

- ▶ **Password Verification**

Password 자체 대신 Password로부터 계산한 Hash Value를 이용한다.

- ▶ **File Integrity Checking**

파일의 Hash Value를 저장해 두고, 나중에 다시 계산하여 파일이 변경되었는지 확인한다.

- ▶ **Digital Signatures**

전체 Message 대신 Message Digest를 이용하여 효율적으로 Digital Signature를 생성할 수 있다.

핵심

Hash Function은 Data의 Compact Fingerprint를 제공하기 때문에 다양한 Security Mechanism의 기본 도구로 사용된다.

7-1. Hash Functions and Passwords

Password P 자체를 저장하는 대신 Hash Value를 저장한다고 생각해 보자.

Registration:

$$P \xrightarrow{H} H(P)$$

Stored:

$$H(P)$$

Login 시 사용자가 입력한 Password P' 에 대해

$$H(P')$$

를 계산하고 Stored Hash와 비교한다.

$$H(P') \stackrel{?}{=} H(P)$$

핵심

Password Verification에서는 입력된 Password가 동일한 Hash를 만드는지 확인하여 Password를 검증할 수 있다.

7-2. Hash Functions and File Integrity

파일이 변경되었는지 확인하기 위해 Hash Function을 사용할 수 있다.

원래 File F 에 대해

$$h = H(F)$$

를 계산하여 안전하게 저장한다.

나중에 File F' 에 대해

$$h' = H(F')$$

를 다시 계산한다.

그리고

$$h' \stackrel{?}{=} h$$

를 비교한다.

Hash Value가 다르면 File이 변경되었다는 것을 알 수 있다.

핵심

Hash Function은 File의 Fingerprint를 생성하여 Data Integrity를 확인하는 데 사용할 수 있다.

8. Encryption, MAC, and Hash

지금까지 배운 Cryptographic Tool을 비교해보자.

Different Cryptographic Tools

	Encryption	MAC	Hash
Secret Key	Yes	Yes	No
Confidentiality	Yes	No	No
Integrity	Not necessarily	Yes	Can detect changes
Source Authentication	Not necessarily	Yes	No
Typical Output	Ciphertext	Authentication Tag	Message Digest

각 Cryptographic Tool은 서로 다른 Security Goal을 위해 사용된다.

핵심

Encryption, MAC, Hash는 비슷해 보일 수 있지만 목적과 Security Property가 서로 다르다.

Chapter 2. Cryptographic Tools

2.3 Public-Key Encryption

Public Key and Private Key

Learning Objectives

이번 Section에서는 Public-Key Encryption의 기본 구조와 주요 개념을 학습한다.

Learning Goals

- ▶ Symmetric Encryption과 Public-Key Encryption의 차이를 이해한다.
- ▶ Public Key와 Private Key의 역할을 이해한다.
- ▶ Public-Key Cryptography를 이용한 Confidentiality 구조를 이해한다.
- ▶ Public-Key Cryptography와 Authentication의 관계를 이해한다.
- ▶ Public-Key Cryptography의 주요 Requirements를 이해한다.
- ▶ RSA, Diffie-Hellman, DSS, ECC의 기본 역할을 구분한다.

1. From Symmetric to Public-Key Cryptography

Symmetric Encryption에서는 Sender와 Receiver가 동일한 Secret Key를 사용한다.

$$C = E_K(M)$$

$$M = D_K(C)$$

따라서 통신을 시작하기 전에 Sender와 Receiver가 Secret Key K 를 안전하게 공유해야 한다.

Public-Key Cryptography에서는 서로 관련된 두 개의 Key를 사용한다.

Public Key

Private Key

핵심

Public-Key Cryptography는 하나의 Shared Secret Key 대신 서로 다른 Public Key와 Private Key를 사용한다.

1-1. Public Key and Private Key

각 사용자는 서로 관련된 두 개의 Key를 생성한다.

Alice의 Key Pair를 다음과 같이 나타내자.

$$(PU_A, PR_A)$$

▶ **Public Key PU_A**

다른 사람들이 사용할 수 있도록 공개한다.

▶ **Private Key PR_A**

Alice만 알고 있어야 하며 외부에 공개하지 않는다.

두 Key는 서로 관련되어 있지만,

$$PU_A \neq PR_A$$

즉 Public Key로부터 Private Key를 알아내는 것은 현실적으로 어려워야 한다.

핵심

Public Key는 공개해도 되지만, Private Key는 Key Owner만 안전하게 보호해야 한다.

1-2. Symmetric vs. Public-Key Cryptography

Symmetric vs. Public-Key

	Symmetric	Public-Key
Number of Keys	One Shared Key	Two Related Keys
Key Type	Secret Key	Public / Private Key
Encryption / Decryption	Same Secret Key	Different Keys
Key Distribution	Secret Key must be shared	Public Key can be distributed
Performance	Generally Fast	Generally More Expensive

핵심

Public-Key Cryptography가 Symmetric Encryption을 대체하는 것은 아니다. 두 방식은 서로 다른 장점과 역할을 가진다.

2. Confidentiality with Public-Key Encryption

Bob이 Alice에게 비밀 Message M 을 보내고 싶다고 하자.

Alice는 자신의 Public Key PU_A 를 공개한다.

Bob은 Alice의 Public Key로 Message를 암호화한다.

$$C = E_{PU_A}(M)$$

Alice는 자신의 Private Key로 복호화한다.

$$M = D_{PR_A}(C)$$

핵심

Confidentiality를 제공하려면 Receiver의 Public Key로 Encrypt하고 Receiver의 Private Key로 Decrypt한다.

2-1. Why Does This Provide Confidentiality?

Alice의 Public Key PU_A 는 누구나 알고 있다.

따라서 공격자 Eve도

$$PU_A$$

를 알고 있으며 Ciphertext

$$C = E_{PU_A}(M)$$

도 관찰할 수 있다.

하지만 Message를 복구하려면

$$PR_A$$

가 필요하다.

Private Key PR_A 는 Alice만 가지고 있어야 한다.

핵심

Public Key와 Ciphertext를 모두 알고 있어도 Private Key 없이는 Plaintext를 현실적으로 복구할 수 없어야 한다.

2-2. What Happens to the Key Distribution Problem?

Symmetric Encryption에서는 Alice와 Bob이 통신하기 전에

K

를 안전하게 공유해야 했다.

Public-Key Cryptography에서는 Alice가 자신의 Key Pair를 Local하게 생성한다.

(PU_A, PR_A)

Alice는

PU_A

만 공개하면 된다.

Private Key PR_A 는 Alice의 시스템 밖으로 전달할 필요가 없다.

핵심

Public-Key Cryptography에서는 Private Key 자체를 상대방과 공유할 필요가 없다.

2-3. Is Key Distribution Completely Solved?

Public Key를 공개할 수 있다고 해서 Key Distribution 문제가 완전히 사라지는 것은 아니다.

Bob이 어떤 Public Key를 받았다고 하자.

Is this really Alice's Public Key?

공격자 Eve가 자신의 Public Key를 보내면서

"This is Alice's Public Key."

라고 속일 수도 있다.

따라서 Public Key의 Owner가 누구인지 확인할 수 있는 Mechanism이 필요하다.

핵심

Public-Key Cryptography에서도 Public Key의 Identity와 Authenticity를 확인하는 Key Management가 필요하다.

2-4. (참고) Certificate Authority (CA)

Public Key의 중요한 문제는 Key 자체가 아니라 **누구의 Public Key인지** 확인하는 것이다.

이를 위해 신뢰할 수 있는 Third Party인 Certificate Authority (CA)를 사용할 수 있다.

Alice는 자신의 Key Pair를 생성한다.

(PU_A, PR_A)

그리고 CA는 Alice의 Identity와 Public Key PU_A 의 관계를 확인한다.

Alice's Identity $\longleftrightarrow$ PU_A

$\Downarrow$

CA Verification

참고

CA의 역할은 Public Key를 만드는 것이 아니라, 특정 Public Key가 누구의 것인지 확인하고 보증하는 것이다.

2-5. (참고) CA Signs Alice's Certificate

CA는 Alice의 Identity와 Public Key를 포함하는 Certificate를 생성한다.

개념적으로,

$$Cert_A = (\text{Alice's ID}, PU_A, \text{Validity}, \dots)$$

CA는 Certificate의 Hash를 계산한다.

$$h = H(Cert_A)$$

그리고 자신의 Private Key PR_{CA} 를 이용하여 Digital Signature를 생성한다.

$$S_{CA} = \text{Sign}_{PR_{CA}}(h)$$

따라서 Signed Certificate는 개념적으로

$$\boxed{Cert_A \parallel S_{CA}}$$

가 된다.

참고

CA의 Signature는 "This Public Key belongs to Alice"라는 관계를 검증할 수 있도록 한다.

2-6. (참고) Verifying Alice's Certificate

Bob은 Alice의 Certificate를 받는다.

먼저 Certificate 내용의 Hash를 계산한다.

$$h_1 = H(\text{Cert}_A)$$

그리고 신뢰하고 있는 CA의 Public Key PU_{CA} 로 CA의 Signature를 검증한다.

$$h_2 = \text{Verify}_{PU_{CA}}(S_{CA})$$

두 값을 비교한다.

$$h_1 \stackrel{?}{=} h_2$$

검증에 성공하면 Bob은 Certificate 안의 PU_A 가 Alice와 연결되어 있음을 신뢰할 수 있다.

참고

Bob은 CA의 Signature를 검증하여 Alice의 Identity와 Public Key의 관계를 확인한다.

2-7. (참고) Where Does the Trust Begin?

그러면 새로운 질문이 생긴다.

How does Bob know the CA's Public Key is genuine?

CA의 Public Key가 단순히 공개되어 있다는 것만으로는 신뢰할 수 있는 것은 아니다.

Bob의 System은 신뢰할 수 있는 Root CA의 Certificate / Public Key를 미리 Trust Store에 가지고 있을 수 있다.



이러한 신뢰의 연결을 Chain of Trust라고 한다.

참고

Certificate System은 Trust 문제를 없애는 것이 아니라, 미리 신뢰하는 CA를 기준으로 Public Key에 대한 Trust를 연결한다.

3. Authentication with Public-Key Cryptography

이번에는 Confidentiality가 아니라 Message의 Source를 확인하고 싶다고 하자.

Bob이 자신의 Private Key PR_B 를 사용한다.

$$Y = E_{PR_B}(M)$$

Alice는 Bob의 Public Key PU_B 를 사용한다.

$$M = D_{PU_B}(Y)$$

Bob의 Private Key는 Bob만 가지고 있으므로, 이 구조는 Message의 Source를 확인하는 Authentication과 연결된다.

핵심

Sender만 가지고 있는 Private Key를 사용한다는 점이 Authentication의 기본 아이디어이다.

3-1. Confidentiality vs. Authentication

Public-Key Cryptography에서는 Security Goal에 따라 사용하는 Key가 달라진다.

Key Usage

Goal	Sender Uses	Receiver Uses
Confidentiality	Receiver's Public Key	Receiver's Private Key
Authentication	Sender's Private Key	Sender's Public Key

Confidentiality에서는

Only the Receiver has the required Private Key.

Authentication에서는

Only the Sender has the required Private Key.

핵심

Confidentiality는 Receiver의 Key Pair, Authentication은 Sender의 Key Pair와 연결된다.

3-2. Connection to Digital Signatures

Sender의 Private Key를 사용하는 구조는 Digital Signature의 기본 아이디어로 연결된다.

Bob's Private Key



Create Signature

Bob's Public Key



Verify Signature

실제 Digital Signature에서는 Message 전체를 단순히 Private Key로 Encrypt하기보다 Message의 Hash와 Public-Key Algorithm을 결합한다.

이 구조는 다음 Section에서 자세히 살펴본다.

핵심

Private Key는 Signature 생성에 사용하고, Public Key는 Signature 검증에 사용할 수 있다.

4. Common Misconceptions

Public-Key Cryptography에 대해서는 몇 가지 흔한 오해가 있다.

1. **Public-Key Encryption is always more secure.**

Public-Key라는 이유만으로 Symmetric Encryption보다 더 안전한 것은 아니다.

2. **Public-Key Cryptography replaces Symmetric Encryption.**

Public-Key와 Symmetric Cryptography는 서로를 완전히 대체하는 관계가 아니다.

3. **Key Distribution becomes trivial.**

Public Key를 공개할 수 있다고 해서 Key Distribution 문제가 완전히 사라지는 것은 아니다.

핵심

Public-Key Cryptography는 새로운 Security Mechanism을 제공하지만, 기존의 모든 문제를 자동으로 해결하는 것은 아니다.

4-1. Is Public-Key Always More Secure?

Public-Key Cryptography가 항상 Symmetric Cryptography보다 더 안전한 것은 아니다.

Security는 단순히 “Public-Key인가 Symmetric-Key인가”에 의해 결정되지 않는다.

실제 Security는 다음과 같은 요소에 의해 결정된다.

- ▶ 사용되는 Algorithm
- ▶ 적절한 Key Size
- ▶ 안전한 Protocol Design
- ▶ 올바른 Implementation
- ▶ Key를 안전하게 관리하는 Key Management

예를 들어, 잘 설계된 Symmetric Encryption은 매우 높은 Security를 제공할 수 있다.

핵심

Public-Key라는 특성 자체가 더 높은 Security를 의미하는 것은 아니다.

4-2. Does Public-Key Replace Symmetric Encryption?

Public-Key Cryptography가 등장했다고 해서 Symmetric Encryption이 필요 없어지는 것은 아니다.

Public-Key Operation은 일반적으로 Symmetric-Key Operation보다 계산 비용이 크다.

따라서 실제 시스템에서는 두 방식을 함께 사용할 수 있다.



핵심

Public-Key와 Symmetric Cryptography는 경쟁 관계라기보다 서로의 장점을 활용하는 Complementary Relationship으로 볼 수 있다.

4-3. Does Key Distribution Become Trivial?

Public Key는 이름 그대로 다른 사람에게 공개할 수 있다.

따라서 처음에는 다음과 같이 생각할 수 있다.

“Public Key를 그냥 공개하면 되므로
Key Distribution 문제는 해결된 것 아닌가?”

하지만 중요한 문제가 남아 있다.

Bob이 Alice의 Public Key라고 주장되는 어떤 Public Key를 받았다고 하자.

Is this really Alice's Public Key?

공격자 Eve가 자신의 Public Key를 보내면서

“This is Alice's Public Key.”

라고 속일 수도 있다.

핵심

Public Key의 배포 자체는 쉬워질 수 있지만, 그 Public Key의 Owner / Identity / Authenticity를 확인하기 위한 Mechanism은 여전히 필요하다.

5. Requirements for Public-Key Cryptography

Public-Key Cryptography가 실용적으로 동작하려면 정상적인 사용자가 수행하는 계산은 쉬워야 한다.

1. Key Pair Generation

$$(PU_B, PR_B)$$

2. Encryption

$$C = E(PU_B, M)$$

3. Decryption

$$M = D(PR_B, C)$$

핵심

Key Generation, Encryption, Decryption은 정상적인 사용자에게 Computationally Easy해야 한다.

5-1. What Should Be Difficult?

반대로 공격자가 수행하려는 계산은 현실적으로 어려워야 한다.

Public Key가 주어져도

$$PU_B \not\Rightarrow PR_B$$

Private Key를 알아내기 어려워야 한다.

Public Key와 Ciphertext를 모두 알고 있어도

$$PU_B, C \not\Rightarrow M$$

원래 Message를 복구하기 어려워야 한다.

핵심

정상적인 사용자가 수행하는 계산은 쉽지만, 공격자가 필요한 역방향 계산은 Computationally Infeasible해야 한다.

6. Applications of Public-Key Cryptography

Public-Key Cryptography는 여러 가지 Security Application에 사용된다.

- ▶ **Encryption**

Confidentiality를 제공한다.

- ▶ **Digital Signature**

Source Authentication과 Integrity를 제공한다.

- ▶ **Key Distribution / Key Agreement**

Symmetric Encryption에 사용할 Secret Key를 설정하는 데 사용한다.

모든 Public-Key Algorithm이 이 모든 기능을 제공하는 것은 아니다.

핵심

Public-Key Algorithm마다 지원하는 Cryptographic Application이 다르다.

6-1. RSA

RSA는 대표적인 Public-Key Cryptographic Algorithm이다.

RSA는 Public Key와 Private Key의 Pair를 사용한다.

Confidentiality를 위한 기본 구조는

$$C = E_{PU}(M)$$

$$M = D_{PR}(C)$$

와 같이 생각할 수 있다.

RSA는 Public-Key Encryption뿐 아니라 Digital Signature와 관련된 Application에도 사용될 수 있다.

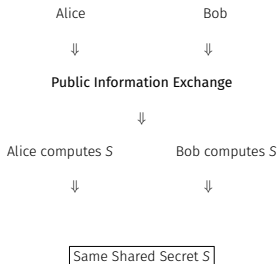
핵심

RSA는 Encryption과 Digital Signature 등에 활용할 수 있는 대표적인 Public-Key Cryptosystem이다.

6-2. Diffie-Hellman Key Agreement

Diffie-Hellman의 목적은 Message 자체를 암호화하는 것이 아니다.

Alice와 Bob이 Network를 통해 정보를 교환하여 **동일한 비밀값(Shared Secret)**을 얻도록 하는 것이다.



중요한 점은 Shared Secret S 자체를 Network를 통해 전송하지 않는다는 것이다.

핵심

Diffie-Hellman은 Secret을 직접 전송하는 것이 아니라, 공개된 정보를 교환하여 양쪽이 동일한 Shared Secret을 계산하도록 한다.

6-3. Shared Secret vs. Secret Key

그렇다면 Diffie-Hellman에서 생성된 **Shared Secret**은 Secret Key와 같은 것인가?

엄밀하게는 구분할 필요가 있다.

- ▶ **Shared Secret**

Diffie-Hellman을 통해 Alice와 Bob이 공동으로 계산한 비밀값

S

- ▶ **Symmetric Secret Key**

AES와 같은 Symmetric Encryption에서 실제로 사용하는 Cryptographic Key

K

일반적으로 Shared Secret S 를 그대로 사용하는 것이 아니라, 이를 기반으로 실제 Secret Key K 를 생성한다.

Shared Secret S $\implies$ Symmetric Secret Key K

핵심

Shared Secret은 두 사용자가 공유하게 된 비밀값이고, 이를 기반으로 실제 Encryption에 사용할 Secret Key를 만들 수 있다.

6-4. From Shared Secret to Secret Key

실제 Protocol에서는 Diffie-Hellman으로 얻은 Shared Secret을 그대로 AES Key로 사용하는 것보다,

Key Derivation Function (KDF)을 이용하여 실제 암호화에 필요한 Secret Key를 생성하는 방식이 일반적이다.



$$K = \text{KDF}(S)$$



즉, Diffie-Hellman은 Shared Secret S 를 만들고, KDF는 S 로부터 실제 Symmetric Encryption에 사용할 Secret Key K 를 생성한다.

핵심

Diffie-Hellman $\rightarrow S \rightarrow \text{KDF}(S) \rightarrow K \rightarrow \text{Symmetric Encryption}$

6-5. Why Diffie-Hellman?

Symmetric Encryption을 사용하려면 Alice와 Bob이 동일한 Secret Key를 가져야 한다.

$$K_A = K_B = K$$

가장 단순한 방법은 Alice가 K 를 생성한 후 Bob에게 보내는 것이다.

$$\text{Alice} \xrightarrow{K} \text{Bob}$$

하지만 Network를 통해 Secret Key 자체를 전송하면 공격자가 이를 획득할 위험이 있다.

Diffie-Hellman의 핵심 아이디어는 다르다.

Secret 자체를 전송하지 않는다.

Public Information만 교환



Alice와 Bob이 각각 계산



동일한 Shared Secret S

핵심

Diffie-Hellman은 Shared Secret 자체를 Network로 보내지 않고도 두 사용자가 동일한 비밀값을 얻을 수 있도록 하는 Key Agreement Algorithm이다.

6-6. Elliptic Curve Cryptography (ECC)

Elliptic Curve Cryptography (ECC)는 Elliptic Curve의 수학적 구조를 이용하는 Public-Key Cryptography이다.

ECC의 중요한 특징은 비교적 작은 Key Size로 높은 Security를 제공할 수 있다는 점이다.

Smaller Key Size



Lower Processing and Storage Overhead

ECC 기반의 Cryptographic Scheme은 Digital Signature, Key Agreement 등 다양한 Public-Key Application에 사용될 수 있다.

핵심

ECC의 주요 장점은 비교적 작은 Key Size로 Public-Key Cryptography를 구성할 수 있다는 것이다.

6-7. Comparing Public-Key Algorithms

Public-Key Algorithm마다 지원하는 Application이 다르다.

Public-Key Applications

Algorithm	Digital Signature	Key Distribution	Encryption
RSA	Yes	Yes	Yes
Diffie-Hellman	No	Yes	No
DSS / DSA	Yes	No	No
Elliptic Curve	Yes	Yes	Yes

따라서 “Public-Key Algorithm”이라는 하나의 이름 아래에서도 각 Algorithm의 목적과 기능은 다를 수 있다.

핵심

어떤 Public-Key Algorithm을 사용할지는 필요한 Security Function에 따라 결정된다.

7. Putting It Together

Public-Key Cryptography에서는 누구의 어떤 Key를 사용하는지가 중요하다.

Public-Key Cryptography

Goal	Basic Idea
Confidentiality	Receiver's Public Key → Receiver's Private Key
Authentication / Signature	Sender's Private Key → Sender's Public Key
Key Agreement	Establish a Shared Secret

이러한 기능은 이후

- ▶ Digital Signature
- ▶ Public-Key Certificate
- ▶ Symmetric Key Distribution

과 같은 실제 Security Mechanism으로 연결된다.

핵심

Public-Key Cryptography는 Confidentiality뿐만 아니라 Authentication과 Key Management를 위한 기본 도구로 사용된다.

8. What We Learned

이번 Section에서는 Public-Key Cryptography의 기본 구조를 살펴보았다.

- ▶ Public-Key Cryptography는 Public Key와 Private Key의 Pair를 사용한다.
- ▶ Public Key는 공개할 수 있지만 Private Key는 안전하게 보호해야 한다.
- ▶ Confidentiality에서는 Receiver의 Public Key로 Encrypt하고 Receiver의 Private Key로 Decrypt한다.
- ▶ Sender의 Private Key 사용은 Authentication과 Digital Signature의 아이디어로 연결된다.
- ▶ Public-Key Cryptography에서도 Public Key의 Identity를 확인하는 Key Management가 필요하다.
- ▶ Public-Key Cryptography는 Symmetric Encryption을 완전히 대체하지 않는다.
- ▶ RSA, Diffie-Hellman, DSS/DSA, ECC는 서로 다른 목적과 기능을 가진다.

핵심

Public-Key Cryptography는 두 개의 서로 관련된 Key를 이용하여 Confidentiality, Authentication, Digital Signature, Key Distribution을 가능하게 하는 기반 기술이다.

Thank You

Questions?